

P1629R1

Transcoding the - Standard Text Encoding

Published Proposal, 2020-03-02

Authors:

[JeanHeyd Meneide](#)

[Shepherd \(Shepherd's Oasis\)](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

?

Latest:

https://thephd.github.io/vendor/future_cxx/papers/d1629.html

Abstract

The standard lacks facilities for transcoding text from one form into another, leaving a serious barrier to entry for individuals who want to process text in any sensible manner in the Standard Library. This paper explores and proposes a static interface for encoding that can be used and built upon for the creation of higher-level abstractions.

Table of Contents

1 Revision History

1.1 Revision 1 - March 2nd, 2020

1.2 Revision 0 - June 17th, 2019

2 Motivation

2.1 The Basic Ideas

- 2.1.1 Reading "Execution Encoding" Data
- 2.1.2 Networking with Boost.Beast
- 2.2 Current Problems
- 2.3 Statement of Objectives

3 Design

- 3.1 Definitions
- 3.2 Low-Level
 - 3.2.1 Error Codes
 - 3.2.2 Result Types
 - 3.2.2.1 Input and Output Ranges
 - 3.2.2.2 Error Handling: Allow All The Options
 - 3.2.3 The Encoding Object
 - 3.2.3.1 Encodings Provided by the Standard
 - 3.2.3.2 UTF Encodings: variants?
 - 3.2.3.3 Encoding Schemes: Byte-Based
 - 3.2.3.4 Default Encodings
 - 3.2.4 Stateful Objects, or Stateful Parameters?
- 3.3 High Level
 - 3.3.1 Eager Free Functions
 - 3.3.1.1 Free Function `decode`
 - 3.3.1.2 Free Function `encode`
 - 3.3.1.3 Free Function `transcode`
 - 3.3.1.4 Free Function `validate`
 - 3.3.1.5 Free Functions `decode_count` and `encode_count`
 - 3.3.2 Safety with the Free Functions
 - 3.3.3 Improving Usability for Low-Memory Environments: Ranges
 - 3.3.3.1 `decode_view` and `decode_iterator`
 - 3.3.3.2 `encode_view` and `encode_iterator`
 - 3.3.3.3 `transcode_view` and `transcode_iterator`
- 3.4 The Need for Speed
 - 3.4.1 Speed and Flexibility for Everyone: Customization Points
 - 3.4.1.1 One-by-one Transcoding Shortcuts
 - 3.4.1.2 Customizability: Transcoding Free Functions
 - 3.4.1.3 Customizability: Validating and Counting Free Functions

4 Implementation Experience

4.1 Previous Work

4.2 Current Work

5 FAQ

- 5.1 Question: Why is there a `max_code_points` value? Won't you only ever output a single unicode code point?
- 5.2 Question: What about Old Unicode Encodings / Private Use Area Encodings?
- 5.3 Question: It can be faster to bulk-decode, then bulk-encode instead of one-by-one transcoding. Why not that design?
- 5.4 Question: Where is the specification for `normalization_view<nfkc>` and `normalize(...)`?
- 5.5 Question: Where is the specification for `std::text::basic_text` and `std::text::basic_text_view`?

6 Acknowledgements

References

Informative References

"In these meetings, these conferences, we only see a little. C++ is not done in the light. The majority of C++ is not done publicly. Most C++ is done privately, in the dark, and that is where it matters most."

– Daniela K. Engert, November 14th, 2019

§ 1. Revision History

§ 1.1. Revision 1 - March 2nd, 2020

- Thoroughly improve [§ 2 Motivation](#).
- Explicit state goals and non-goals in the [§ 2.3 Statement of Objectives](#).
- Rewrite most of paper to more thoroughly explain the API, especially the [§ 3.3 High Level](#) section with `validate`, `decode_count`, `encode_count`, and more APIs.
- Include drastically improve the explanation for the free functions in [§ 3.3.1 Eager Free Functions](#).

- Emphasize the need for ranges in [§ 3.3.3 Improving Usability for Low-Memory Environments: Ranges](#).
- Add new descriptions in the low-level API regarding error handling in [§ 3.2.2.2 Error Handling: Allow All The Options](#).
- Describe customization points in full in [§ 3.4.1 Speed and Flexibility for Everyone: Customization Points](#).
- The [Implementation](#) is now hidden, after [doing a magic trick](#). Contact the author for access.
- Add [§ 5 FAQ](#).
- Going no-where, targeted at no-one.

§ 1.2. Revision 0 - June 17th, 2019

- Initial release of exploratory paper.

§ 2. Motivation

It's 2020 and Unicode is still barely supported in both the C and C++ standards.

From the POSIX standard requiring a single-byte encoding by default, heavy limitations placed in `codecvt` facets in C and C++, and the utter lack of UTF8/16/32 multi-unit conversion functions by the standard, the programming languages that have shaped the face of development in operating systems, embedded devices and mobile applications has pushed forward a world that is incredibly unfriendly to a world of text beyond ASCII English. Developers frequently roll their own solutions, and almost every major codebase -- from Chrome to Firefox, Qt to Copperspice, and more -- all have their own variations of hand-crafted text processing. With no standard implementation in C++ and libraries split between various third party implementations plus ICU, it is increasingly difficult and error-prone to handle what is the basic means of communication between people on the planet using C++.

This paper aims to explore the design space for both extremely high performing transcoding (encoding and decoding) as well as a flexible one-by-one interface for more careful and meticulous text processing. This proposal arises from industry experience in large codebases and best-practice open source explorations with [\[libogonek\]](#), [\[icu\]](#), [\[boost.text\]](#) and [\[text_view\]](#) while also building on the concepts and design choices found in both [\[range-v3\]](#) and pre-existing text encoding solutions such as Windows's `WideCharToMultiByte` interfaces, *nix utility `iconv`, and more.

The ultimate goal is to allow an interface that is correct by default but capable of being fast both by Standard Library implementer efforts but also program overridable ADL free functions. It will produce interfaces for encoding, decoding, and transcoding in eager and lazy forms.

§ 2.1. The Basic Ideas

While some of these types aren't contained in this paper, the end goal is to enable the following to be possible:

```
#include <encoding> // this proposal
#include <text>      // future proposal

int main (int, char*[]) {
    using namespace std::literals;
    std::text::u8text my_text
        = std::text::transcode("안녕하세요 🙌"sv, std::text::utf8{});
    std::cout << my_text << std::endl; // prints 안녕하세요 🙌 to a capab
    std::cout << std::hex;
    for (const auto& cp : my_text) {
        std::cout << static_cast<uint32_t>(cp) << " ";
    }
    // 0000c548 0000b155 0000d558 0000c138 0000c694 00000020 0001f44b
    return 0;
}
```

This paper is in support of reaching this goal. The following examples are more concretely tied to this proposal in particular.

§ 2.1.1. Reading "Execution Encoding" Data

The following is an example of opening a file handle on Windows after converting from the execution encoding of the system `argv` to the wide arguments for `CreateFileW`.

```

#define WINDOWS_LEAN_AND_MEAN 1
#include <windows.h>

#include <encoding> // this proposal
#include <iostream>

int main (int argc, char* argv[]) {

    if (argc < 2) {
        std::cerr << "Path unspecified: exiting." << std::endl;
        return -1;
    }

    std::wstring path_as_wstr = std::text::transcode(
        std::string_view(argv[1]), std::text::wide_execution{});

    // Interop with Windows
    std::unique_ptr<HANDLE, FileHandleDeleter> target_file =
        CreateFileW(path_as_wstr.data(), GENERIC_WRITE,
            0, NULL, CREATE_ALWAYS,
            FILE_ATTRIBUTE_NORMAL);

    if (!target_file) {
        // GetLastError(), etc...
        return -2;
    }

    /* Use File... */

    return 0;
}

```

This paper directly enables such a use case.

§ 2.1.2. Networking with Boost.Beast

The following is an example using this proposal to do a byte-based read off the network of a UTF-16 Big Endian payload in any machine.

```

#include <boost/beast.hpp>
#include <boost/beast/http.hpp>
#include <boost/asio/ip/tcp.hpp>

#include <iostream>
#include <encoding> // this proposal

namespace beast = boost::beast;
namespace http = beast::http;
using tcp = boost::asio::ip::tcp;
using results_type = tcp::resolver::results_type;

class session : public std::enable_shared_from_this<session> {
    /* ... */
    http::request<http::empty_body> req_;
    std::vector<std::byte> res_body_;
    http::response<http::vector_body<std::byte:> res_;
    std::u8string converted_body_;

    /* ... */

    void on_connect(beast::error_code ec, results_type::endpoint_type);
    void on_resolve(beast::error_code ec, results_type results);

    /* ... */

    void on_read(beast::error_code ec, std::size_t bytes_transferred) {
        if (ec) {
            log_fail(ec, u8"read failed");
            return;
        }

        std::span<std::byte> bytes(res_body_.data(), bytes_transferred);
        std::ranges::unbounded_view output(std::back_inserter(converted_body_),
            bytes);

        // utf16, but big endian
        std::text::encoding_scheme<
            std::text::utf16,
            std::endian::big
        > from_encoding{};

        std::text::utf8 to_encoding{};
    }
};

```

```

        // transcode from bytes that are UTF16, Big Endian,
        // into unbounded output
        std::text::transcode(bytes, output, from_encoding, to_encoding);
        std::clog << converted_body_ << std::endl;

        /* Commit / clean up, etc. */
    }
};

```

This paper directly enables such a use case.

§ 2.2. Current Problems

I don't write any software which runs only in English. I'm tired of writing the same code different ways all the time just to display a handful of strings. Lately, I just skip C++ for anything that displays UI -- it's so much easier in every other modern language.

This is REQUIRED for using C++ with any software which needs to run in multiple languages, without rolling your own code. I'm tired of writing this from scratch for every separate project (cannot share code for most of them), using different underlying libraries for each (as licensing and processing requirements vary, I can't just pick one library and use it everywhere). Unfortunately, I have no confidence the ISO committee understands the problem well enough, given how it patted itself on the back so much for adding `u8""`, `u""`, and `U""` a while back. Real-world software which runs in multiple languages never hard-codes strings...

Norway has its own character set which is a variant of ISO-8859-10 with modifications to a couple of characters. This proposal would ease the transition for existing software when C++ gets (better/more coherent) support for Unicode.

The standard : "Oh yeah hey dudes `codecvt` is deprecated but we didn't feel like writing an alternative so good luck yolo".

– [Herb Sutter's "Top 5 C++ Proposals" Survey, Survey Respondent](#)

Text in the Standard is a desert wasteland.

After pulling `std::wstring_convert` from the language (for a very good reason, yes), users were left with no proper utilities to convert Unicode to Unicode, or convert execution / wide execution text to Unicode and back. People reach out for ICU, but the API -- while extremely fast -- is opaque and not the friendliest to use. `iconv` is not easy to build everywhere, and applications ages ago have shipped all manner of ad-hoc solutions (or not) to the text problem without working together or sharing their libraries with the whole ecosystem. As text -- and particularly, the encoding of text -- stands as one of the greatest barriers to Systems Programming languages being more diverse and friendly, there is a strong obligation to provide a standard solution that is capable of lasting the next 40 years unmodified.

The use cases for text encoding are vast. From: basic processing of user-entered data; sanitization of scripts; domain name protection in browsers; text conversions when working with legacy systems or differing new/Unicode systems; supplying the components that can be successfully used with industry-standard FreeType/HarfBuzz and DirectWrite; talking properly to legacy GDI applications; communicating string data in JSON; receiving market data from the Chinese Exchange in GB18030; converting and preserving government data in digital records; handling data generated by logs in a multitude of languages; handling user names without mangling; and hundreds of dozens of other use cases, the need for text practically writes itself.

§ 2.3. Statement of Objectives

Part of this proposal is identifying exactly how those needs should be served. The primary objectives of this proposal, therefore, is as follows:

- Users should be able to define their own encodings for their own encodings. Jonathan Wakely's time is not worth EBCDIC, but IBM will certainly be very invested in making sure EBCDIC and its code pages is well-implemented and optimized. Put another way: company-specific and user-specific problems should be specific to them and not exported to the whole ecosystem, and they should be able to handle their problems effectively and efficiently without throwing the C++ Standard in the trash.
- Locale-based `char` and `wchar_t` encodings belong to the C and C++ implementation. If users need to guess about the locale's encoding and (probably extremely wrongly) pick something rather than using this API, then the API is a failure.
- The standard library should be able to cannibalize all existing legacy encodings and -- by way of leading design -- encourage and promote the use of Unicode in the user's code.
- The standard library (and its implementers) do not have time to implement every new, old, and existing encoding. Put bluntly: CJ Johnson's brilliance and Stephan T. Lavavej's passion is better

spent improving their respective libraries and fixing bugs, not implementing EBCDIC or ISO/IEC 2022 CN, extended variant 2.

- Unicode is the one and only language the standard speaks in its higher level text algorithms and functionality: legacy encodings must convert to Unicode to work with functionality built beyond this proposal. Future proposals will never need to concern themselves with encodings after this proposal is done.
- Users may choose not to convert to Unicode, but they will need to spend the time and effort working out that trade off with their environment. The standard library will never have to care about text that willingly and deliberately exits the Unicode system.
- Safety is not optional. Code that performs unsafe operations should require explicit opt-in and easily searchable patterns and names that make it clear the user has made a deliberate choice to open themselves up to vulnerabilities such as Undefined Behavior.
- Performance is not optional, and [correctness isn't a tender suggestion achievable with insane workarounds](#).
- Simple function calls should be simple, but if the user wants to pry open the details they should be able to do so incrementally with ease.
- Nobody has time to reimplement all of iconv, especially the library developers. The interface should allow implementers to substitute a backend for certain encodings that takes advantage of pre-existing Operating System, Widely-Available Library, or similar functionality.
- Users should be able to do everything implementers can without undue clash between user functionality and implementer internal handling and extensions.
- Octets -- delivered over the network, from IPC, or similar -- are an important input case that must be handled.
- The design must be viable for low-memory environments, and prioritize zero allocation if a user cares enough to invest the time into the API with that goal.
- At no point should we be introducing new container types for this functionality. Container wrappers / adaptors and range wrapper / adaptors are enough.

§ 3. Design

The current design has been the culmination of a few years of collaborative and independent research, starting with the earliest papers from Mark Boyall's [\[n3574\]](#), Tom Honermann's [\[p0244r2\]](#), study of ICU's interface, and finally the musings, experience and work of R. Martinho Fernandes in [\[libogonek\]](#). Current and future optimizations are considered to ensure that fast paths are not blocked in the interface proposed for standardization. With [\[boost.text\]](#) showing an interface with a nailed

down internally used UTF-8 encoding, Markus Sherer’s participation in SG16 meetings, Henri Sivonen’s feedback on blog posts and mailing lists, and Bob Steagall’s [work in writing a fast UTF8 decoder](#) this paper absorbs a wealth of knowledge to get reach a flexible interface that enables high-throughput.

In reading, implementing, working with and consuming all of these designs, the author of this paper, independent implementers, and several SG16 members have come to the following core tenants:

- strong types for code units allow selecting proper default encodings for these interfaces;
- iterators and ranges are a huge interface win for working with text but are impossible to provide the fastest possible way to encode/decode/transcode text;
- and, avoid creating new vocabulary: improve working with original containers and imposing well-formedness constraints upon them rather than designing new containers from the ground up.

Given these tenants, the following interface choices have arisen for this paper. Each section will describe a piece of the interface, its goals, and how it works. A low-level encoding interface and its plumbing and core types will be described first, followed by a high level interface that makes the low level easy to use. Both are imperative to cover the full design space that exists together, and the use cases today.

§ 3.1. Definitions

Some handy definitions here which will be used liberally applied to template parameters and other things to shorten the specification.

- Unicode Code Point: the 21-bit value (often represented as a 32-bit number for implementation-related reasons) that represents a code point from the Unicode Standard. Specifically, it is the range of integers 0 to 0x10FFFF inclusive.
- Unicode Scalar Value: the 21-bit value that represents a code point from the Unicode Standard, but without Surrogate Unicode Code Point values. Specifically, it is the ranges of integers 0 to 0xD7FF and 0xE000 to 0x10FFFF inclusive.
- `unicode_code_point`: a type in C++ that represent at Unicode Code Point. Alias of `char32_t`.
- `unicode_scalar_value`: a type for C++ that represents. Strong typedefs that supports all the same operations as `char32_t`.
- `using UEncoding = std::remove_cvref_t<Encoding>` given the existence of a template parameter `Encoding`.

- `using UToEncoding = std::remove_cvref_t<FromEncoding>` given the existence of a template parameter `FromEncoding`.
- `using UFromEncoding = std::remove_cvref_t<ToEncoding>` given the existence of a template parameter `ToEncoding`.
- `template <typename T> using encoding_state_t = typename std::remove_cvref_t<T>::state;`
- `template <typename T> using encoding_code_unit_t = typename std::remove_cvref_t<T>::code_unit;`
- `template <typename T> using encoding_code_point_t = typename std::remove_cvref_t<T>::code_point;`; this is the `code_point` type definition for a given type `T`, ignoring cv-qualifiers.
- `is_self_state_encoding_v<T>`: a boolean trait that tells whether or not an encoding uses itself as the state type, rather than a separate state type.

```
template <typename T>
inline constexpr bool is_self_state_encoding_v
    = std::is_same_v<std::remove_cvref_t<T>, encoding_state_t<T>>;
```

- `range_of<T>`: is a concept defining that there is a range whose iterator produces a `value_type` of `T`. For example, `std::vector<int>` and `int[1]` model concept-constrained parameter or return type of `const range_of<int> auto&`.

```
template <typename R, typename T>
concept range_of = std::ranges::range<std::remove_cvref_t<R>> &&
    std::is_same_v<std::ranges::range_value_t<std::remove_cvref_t<R>>,
```

- `contiguous_range_of<T>`: is a concept defining that there is a range whose iterator produces a `value_type` of `T`. For example, `std::span<double>` and `double[1]` model concept-constrained parameter or return type of `const contiguous_range_of<double> auto&`.

```
template <typename R, typename T>
concept contiguous_range_of = std::ranges::contiguous_range<std::remove_cvref_t<R>> &&
    std::is_same_v<std::ranges::range_value_t<std::remove_cvref_t<R>>,
```

§ 3.2. Low-Level

The high-level interfaces must be built on *something*: it cannot be magically willed into existence. There is quite a bit of plumbing that goes into the low-level interfaces, most of which will be boilerplate to users but will serve keen use and importance to several library developers and standard library implementers.

§ 3.2.1. Error Codes

There is some boilerplate that needs to be taken care of before building our encoding, decoding, transcoding and similar functionality begins. First and foremost is the error codes and result types that will go in and out of our encoding functions. The error code enumeration is `std::text::encoding_errc`. It lists all the reasons an encoding or decoding operation can fail:

```

namespace std { namespace text {

    enum class encoding_errc : int {
        // just fine
        ok = 0x00,
        // input contains ill-formed sequences
        invalid_sequence = 0x01,
        // input contains incomplete sequences
        incomplete_sequence = 0x02,
        // output cannot receive all the completed
        // code units
        insufficient_output_space = 0x03,
        // sequence can be encoded but resulting
        // code point is invalid (e.g., encodes a lone surrogate)
        invalid_output = 0x04,
        // input contains overlong encoding sequence
        // (e.g. for utf8)
        overlong_sequence = 0x05,
        // leading code unit is wrong
        invalid_leading_sequence = 0x06,
        // leading code units were correct, trailing
        // code units were wrong
        invalid_trailing_sequence = 0x07
    };

}}

```

The comments give some small amount of examples about what each one means. The reason 0 is used to signal success is very simple: the next part of the API creates an `encoding_error_category` class and hooks up the machinery for a `std::error_condition`:

```

namespace std {

    template <>
    class is_error_condition_enum<encoding_errc> : true_type {};

    class encoding_error_category : public error_category {
    public:
        constexpr encoding_error_category() noexcept;

        virtual const char* name() const noexcept override;
        virtual string message(int condition) const override;
    };

}

```

This allows the creation of a `std::error_condition`, which is used as an all-encompassing text error code for the standard.

§ 3.2.2. Result Types

The result types are the glue that help users who use the low level interface loop through their text properly. It returns updated ranges of both the input and output to indicate how far things have been moved along, on top of an `error_code` and whether or not the result came from an error being handled:

```

namespace std { namespace text {

    template <typename Input, typename Output, typename State>
    class encode_result {
        Input input;
        Output output;
        State& state;
        encoding_errc error_code;
        bool handled_error;

        template <typename InRange, typename OutRange, typename EncodingState>
        constexpr encode_result(InRange&& input, OutRange&& output,
                                EncodingState&& state, encoding_errc error_code = encoding_errc())
            : input(input), output(output), state(state), error_code(error_code) {}

        template <typename InRange, typename OutRange, typename EncodingState>
        constexpr encode_result(InRange&& input, OutRange&& output,
                                EncodingState&& state, encoding_errc error_code, bool handled_error)
            : input(input), output(output), state(state), error_code(error_code), handled_error(handled_error) {}

        constexpr std::error_condition error() const;
    };

    template <typename Input, typename Output, typename State>
    class decode_result {
        Input input;
        Output output;
        State& state;
        encoding_errc error_code;
        bool handled_error;

        template <typename InRange, typename OutRange, typename EncodingState>
        constexpr decode_result(InRange&& input, OutRange&& output,
                                EncodingState&& state, encoding_errc error_code = encoding_errc())
            : input(input), output(output), state(state), error_code(error_code) {}

        template <typename InRange, typename OutRange, typename EncodingState>
        constexpr decode_result(InRange&& input, OutRange&& output,
                                EncodingState&& state, encoding_errc error_code, bool handled_error)
            : input(input), output(output), state(state), error_code(error_code), handled_error(handled_error) {}

        constexpr std::error_condition error() const;
    };

    template <typename Input, typename Output, typename FromState, typename ToState>
    class transcode_result {
        Input input;

```



```

        Output output;
        FromState& state;
        ToState& state;
        encoding_errc error_code;
        bool handled_error;

        template <typename InRange, typename OutRange,
                  typename FromEncodingState, typename ToEncodingState>
        constexpr decode_result(InRange&& input, OutRange&& output,
                                FromEncodingState&& from_state, ToEncodingState&& to_state,
                                encoding_errc error_code = encoding_errc::ok);

        template <typename InRange, typename OutRange,
                  typename FromEncodingState, typename ToEncodingState>
        constexpr decode_result(InRange&& input, OutRange&& output,
                                FromEncodingState&& from_state, ToEncodingState&& to_state,
                                encoding_errc error_code, bool handled_error);

        constexpr std::error_condition error() const;
};

template <typename Input, typename State>
struct validate_result {
    Input input;
    bool valid;
    State& state;

    template <typename ArgInput, typename ArgState>
    constexpr validate_result(ArgInput&& input, bool is_valid,
                              ArgState&& state);
};

template <typename Input, typename State>
struct count_result {
    Input input;
    size_t count;
    State& state;
    encoding_error error_code;
    bool handled_error;

    template <typename ArgInput, typename ArgState>
    constexpr count_result(ArgInput&& input, size_t count, ArgState&& state,
                           encoding_error error_code = encoding_error::ok);
};

```

```

        template <typename ArgInput, typename ArgState>
        constexpr count_result(ArgInput&& input, size_t count, ArgState& state,
                                encoding_error error_code, bool handled_error);
    };

}}

```

There is a lot to unpack here. There are two essentially identical structures:

`std::text::encode_result` and `std::text::decode_result`. These contain the input range, the output range, a reference to the encoding's current state, the error code and whether or not the error handler was invoked. The `bool error_handled` is important because some error handlers may change the `error_code` member to `std::text::encoding_errc::ok`, indicating that things are fine (e.g., a replacement character was successfully inserted into the output stream to replace some bad input).

Note: Having 2 differently-named types with much the same interface is paramount to allow an `error_handler` callable to know how to interpret some errors and whether to try to insert code units into the output stream or code points into the output stream (encoding means code units into output, decoding means code points into the output). If the structures were merged, this information would be lost at compile-time and have to attempt to coerce that information out by examining the `value_type` and `reference` types of the output or input range. Unfortunately, even that is not foolproof because neither the input range or output ranges need to exactly dereference to exactly `Encoding::code_unit` or `Encoding::code_point` types, just things convertible to / from them.

`transcode_result` is a joint type for operations which go from `code_unit` → `code_point` and then `code_point` → `code_unit`, assuming the `code_point` types are compatible between the two encodings deployed for the transformation.

§ 3.2.2.1. *Input and Output Ranges*

These are essentially the ranges moved forward as much or as little as the encoding needed to for reading from the input, converting, and writing to the output. It also solves the problem of obtaining maximal speed based on checking if the destination is filled or if the input is exhausted:

`unbounded_view` works well since its comparison sentinel always returns the literal "false" bool on comparison, meaning that any compiler beyond the typical `-O0` / `/O0` / etc. levels of optimization will cull any `it == last` comparison branches out of code.

The decoding result and encoding result types both return the input and output range passed to encoding and decoding functions in the structure itself. This represents the changed ranges. In the event where the range cannot be successfully reconstructed from itself using the iterator and sentinel, a `std::ranges::subrange<Iterator, Sentinel>` will be returned instead.

§ 3.2.2.2. Error Handling: Allow All The Options

This is a low-level interface. As such, accommodating different error handling strategies is necessary. There are several ways to report errors used in both the C and C++ standard libraries, from throwing errors, to `error_code` out parameters, to integral return values and even complex return structures. Choosing a scheme here is difficult given the large breadth and depth of error handling history in C++, and while the standard library shows a clear bias towards throwing exceptions it would not be prudent to throw all the time. Requiring exceptions may exclude hard and soft real-time programming environments wherein these encoding structures will be needed. Exceptions also have an intrinsic problem in this domain, as described a little bit below in this section.

To accommodate the wide breadth of C++ programming environments and ecosystems, error reporting will be done through an error handler, which can be any type of callable that matches the desired interface. The standard will provide 4 of these error handlers:

```
namespace std { namespace text {  
  
    class replacement_handler;  
    class throw_handler;  
    class assume_valid_handler;  
    class default_handler;  
  
}}
```

The interface for an error handler looks like the below example error handler:

```

namespace std { namespace text {

    class example_error_handler {
    public:
        template <typename Encoding, typename InputRange,
                  typename OutputRange, typename State,
                  contiguous_range_of<encoding_code_point_t<Encoding>
constexpr auto operator()(const Encoding& encoding,
                          encode_result<InputRange, OutputRange, State> result,
                          const Progress& progress) const {
            /* morph result, log, throw error, etc. ... */
            return result;
        }

        template <typename Encoding, typename InputRange,
                  typename OutputRange, typename State,
                  contiguous_range_of<encoding_code_unit_t<Encoding>
constexpr auto operator()(const Encoding& encoding,
                          decode_result<InputRange, OutputRange, State> result,
                          const Progress& progress) const {
            /* morph result, log, throw error, etc. ... */
            return result;
        }

    };

}}

```

The specification here is a value-based one. `encoding` is a reference to the encoding which threw the error. `current_result` is passed to the error handler and it represents an `encode` or `decode` function's current progress. The `result` types provide the current input range, the current output range, a reference to the current state, and the type of error encountered according to the `std::text::encoding_errc`. Finally, the `progress` object is a `std::contiguous_range` passed from the encoder with the code points or code units already read from the input range. (This is important for e.g. reading from one-way iterators like `istream_iterator`, where it is impossible to go back and recover information consumed by the algorithm.) The error handler is then responsible for performing any modifications it wants to the result type, before returning the modified result to be propagated back by the encoding interface.

There are a few things that can be done in the commented code shown above. First and foremost is that someone could look at `current_result.error()` and simply throw a hand-tailored exception. This would bubble out of the function and let the caller decide what to do.

Note: Throwing is **explicitly not recommended by default** by prominent vendors and implementers (Mozilla, Apple, the Unicode Consortium, WHATWG, etc.). Ill-formed text is common. Text from misbehaving programs -- 40 years of them -- is a frequent kind of user and machine input. It is extremely easy to provoke a Denial of Service Attack (DoS Attack) if an application throws an error on malformed input that the application author did not consider.

The default error handler will be the `std::text::default_handler`, as hinted by the name. The `default_handler` is a "strong typedef" over the `std::text::replacement_handler`, done for the purposes of [safety in the higher-level API](#).

The `replacement_handler` will look inside `Encoding` to see if the expression `encoding.replacement_code_points()` or `encoding.replacement_code_units()` is well-formed. If so, it will take the range returned from that function and will attempt to insert it into the `output` range. Specifically:

- On a failure in `decode_one`:
 - If the output is at its end, return the result as-is.
 - If the expression `decltype(auto) replacement_points = encoding.replacement_code_points();` is well-formed, then `replacement_points` is iterated over and code points are inserted into the output range in linear ascending order, if there is space. If there is not enough space, return the result as-is. Note that this may write partial data to the range if `replacement_points` contains more than one code point.
 - Otherwise, if the `code_point` type is a Unicode Code Point type (`char32_t`, `unicode_code_point`, `unicode_scalar_value`), an array of `{ 'U\uFFFD' }` is assumed to be the replacement characters for the standard error handlers.
 - Otherwise, if the expression `decltype(auto) replacement_units = encoding.replacement_code_units();` is well-formed, then `replacement_units` is passed to a call to `auto intermediate_result = encoding.decode_one(replacement_units, result.output, /* implementation-defined pass-through handler */, result.state);`. If `intermediate_result.error_code` is not equal to `std::text::encoding_errc::ok`, then return the original result. Note that this may write partial data to the range if the decode operation needs to write more than one code point to the `output`.
 - Otherwise, the program is ill-formed.
- On a failure in `encode_one`:

- If the output is at its end, return the `result` as-is.
- If the expression `decltype(auto) replacement_units = encoding.replacement_code_units();` is well-formed, then `replacement_units` is iterated over and code points are inserted into the output range in linear ascending order, if there is space. If there is not enough space, return the `result` as-is. Note that this may write partial data to the `result.output` range if `replacement_units` contains more than one unit but the output reaches its limit.
- Otherwise, if the `code_point` type is a Unicode Code Point type (`char32_t`, `unicode_code_point`, `unicode_scalar_value`), an array of `{ 'U\uFFFD' }` is assumed to be the replacement characters for the standard error handlers.
- Otherwise, if the expression `decltype(auto) replacement_points = encoding.replacement_code_points();` is well-formed, then `replacement_points` is passed to a call to `auto intermediate_result = encoding.encode_one(replacement_points, result.output, /* implementation-defined pass-through handler */, result.state);`. If `intermediate_result.error_code` is not equal to `std::text::encoding_errc::ok`, then return the original result. Note that this may write partial data to the range if the encode operation needs to write more than one code point to the `result.output`.
- Otherwise, the program is ill-formed.

If successful, the error code on the result will be corrected to say "everything is fine" (`std::text::encoding_errc::ok`) and then returned from the function. This allows algorithms continue looping over input with the replacement characters inserted. If there is no room in the output, then the error is returned untouched.

For performance reasons and flexibility, the error callable must have a way to ensure that the user and implementation can agree on whether or not Undefined Behavior is invoked by assuming that the text is valid. [\[libogonek\]](#) made an object of type `assume_valid_t`. This paper provides the same here: an error handler of `assume_valid_handler` means that the implementation will eliminate all of its checks and subsequent calls to the error handling interface. A user must provide the `assume_valid_handler` to achieve this behavior: it will never be the default behavior because it is error-prone and dangerous and only to be performed with explicit user consent.

This is notably important: Rust attempted to force that every string constructed ever was valid UTF-8 and rigorously checked this pre- and post-condition. Doing this check was so obscenely expensive that they needed to introduce a new function to `escape(...)` some UTF-8 text so it would not be checked if the user knew the text was in the proper encoding.

§ 3.2.3. The Encoding Object

It is no great surprise that there is not enough library implementers prepared to standardize the entirety of what the WHATWG specifies in its encoding specification, let alone enough to handle every rogue request for a new encoding object type in C++ Standard. A system must be developed that provides flexibility for the end-user that does not require them writing a paper and getting into a 1-2 year long process of herding a proposal through the notoriously slow Committee, just to have support for X encoding or Y feature. There is also less and less (read: almost none) tolerance for adding whacky extension to libraries like libstdc++ or libc++, and MSVC has only recently open-sourced (with no appetite for shoveling more semi-abandonware legacy library extensions into their codebase at the time of writing).

Encoding objects provide flexibility that enable us to consume the entire encoding space without needing to tax the Standard Library. It enables other people to plug into the system and provides the flexibility they need, and only standardize when **interoperability** and **redundant implementation** becomes a burden to the greater C++ ecosystem. This frees up Billy O’Neal, Jonathan Wakely, Louis Dionne, their successors, and the dozens of other standard library contributors and implementers to focus on producing high quality code, rather than scrambling to implementing four or five dozen encodings because one company, somewhere, made an at-the-time-it-seemed-okay choice in 2005 about how to store their text.

Given our result types and error handlers, the interface for the encoding object itself can be defined. Here is the example encoding illustrating the interface:

```

namespace std { namespace text {

    // NOTE: exemplary encoding
    // for expository purposes
    // containing all the types
    class example_locale_encoding {
        class example_state {
            std::mbstate_t multibyte_state;
        };

        // REQUIRED: member types and variables
        using code_point = char32_t;
        using code_unit = char;

        using state = example_state;

        static constexpr size_t max_code_unit_sequence = MB_LEN_MAX;
        static constexpr size_t max_code_point_sequence = 1;

        // OPTIONAL: member types and variables
        using is_encoding_injective = std::false_type;
        using is_decoding_injective = std::true_type;

        // REQUIRED: functions
        template <typename In, typename Out, typename Handler>
        decode_result<In, Out, state> decode(
            In&& in_range,
            Out&& out_range,
            Handler&& handler,
            state& current_state
        );

        template <typename In, typename Out, typename Handler>
        encode_result<In, Out, state> encode(
            In&& in_range,
            Out&& out_range,
            Handler&& handler,
            state& current_state
        );

        // OPTIONAL: functions
        constexpr const range_of<code_point> auto&
        replacement_code_points () const noexcept;
    };
}
}

```



```

        constexpr const range_of<code_unit> auto&
        replacement_code_points () const noexcept;
    };
}}

```

There are many pieces of this encoding object. Some of them fit the purposes explained above. As an overview, given an `Encoding` type such as `example_locale_encoding`, the following type definitions, static member variables, and functions are required:

- `code_unit` and `code_point` type definitions let us know what an Encoding’s inputs and outputs will be from its functions. It also helps us tell if 2 encodings can be transcoded from one another by having at least the `code_point` in common.
- `state` allows a user to instantiate the type and control any parameters for manipulating stateful or shift-state encodings.
 - If `is_encoding_self_state_t<Encoding>` is false (the encoding does not name itself as its state type), `encoding_state_t<Encoding>` must be default-constructible and default construction results in .
 - If `is_encoding_self_state_t<Encoding>` is true (the encoding names itself as its state type), then the encoding may not be default-constructible.
- `max_code_unit_sequence` and `max_code_point_sequence` represent integral values which inform users of the encoding the necessary size of a buffer to handle at least one full, encoded sequence of code units and one full, decoded sequence of code points. In most cases, `max_code_point_sequence` will be 1, but there are cases where this is not the case (e.g., the Tamil Standard Code for Information Interchange (TASCII)).
- `decode` and `encode` are fundamental functions which convert one full unit of complete, indivisible information from one representation to the other. Specifically, `decode` converts from `code_units` to `code_points`, and `encode` converts from `code_points` to `code_units`. `In` is an input range, `Out` is an output range, and `handler` is an error handler as defined in [§ 3.2.2.2 Error Handling: Allow All The Options](#).

Optionally, some additional type definitions and functions help with safety, error handling (for replacement), and more:

- `is_encoding_injective` and `is_decoding_injective` indicate whether or not the encode or decode operations provide a lossless map from the `code_point` to `code_unit` or vice-versa, respectively. This is important when using high-level conversion facilities: [compile-time diagnostics](#) can be issued for conversions that are lossy. This ensures that users who do lossy

conversions must specify an `error_handler` from the standard or one of their own making and know what they are getting into with bad encodings.

- `replacement_code_points` is a function that returns a range to be entered into the output if an error occurs during a `decode` call and the error handler used is the `std::text::default_handler` or `std::text::replacement_handler`. This provides encodings a simple way to plug in replacement code points that are not the same as the default replacement character used is, which is `\uFFFD` (◆). This can be defined to be an empty range (not recommended but possible).
- `replacement_code_units` is a function that returns a range to be entered into the output if an error occurs during an `encode` call and the error handler used is the `std::text::default_handler` or `std::text::replacement_handler`. Note that not all encodings can handle the entirety of the Unicode Code Point space, let alone `\uFFFD` (◆). This can be defined to return an empty range (not recommended, but possible).

§ 3.2.3.1. *Encodings Provided by the Standard*

The primary reason for the standard to provide an encoding is to ensure that it produces a way for applications to communicate with one another. As a baseline, the standard should support all the encodings it ships with its string literal types. On top of that, there is an important base-level optimization when working with strictly ASCII text that can be implemented with UTF8 which would most library implementers are interested in shipping. This means that the following encodings will be shipped by the standard library:

```

// header: <encoding>

namespace std { namespace text {

    using unicode_code_point = char32_t;
    class unicode_scalar_value;

    template <typename CharT>
    class basic_utf8;
    template <typename CharT>
    class basic_utf16;
    template <typename CharT>
    class basic_utf32;

    template <typename Encoding,
              std::endian endianness = std::endian::native,
              typename Byte = std::byte>
    class encoding_scheme;

    class ascii;
    using utf8 = basic_utf8<char8_t>;
    using utf16 = basic_utf16<char16_t>;
    using utf32 = basic_utf32<char32_t>;
    class narrow_literal;
    class wide_literal;
    class narrow_execution;
    class wide_execution;

}}

```

All of `ascii`, `utf8`, `utf16`, `utf32`, `narrow_literal`, and `wide_literal` correspond directly and obviously to what they name. These six encodings are also `constexpr`-capable encodings in that they can be called at compile-time and used inside of contexts with other `constexpr` functions, such as within `static_assert`s.

Both `narrow_execution` and `wide_execution` represent the dynamic locale-based encoding that is used as the default encoding for C library functions. They are key encodings for interoperating with locale-dependent narrow execution encoding data as well as locale-dependent wide execution encoding data. It is imperative the standard ships these because only the implementation knows the runtime narrow or wide execution encoding. `encoding_scheme`'s supremely helpful utility is described [is described below](#).

These represent the core 9 encodings must be shipped with the standard, no matter what.

`ascii` holds a special place here because it is a direct subset of `utf8`. If an individual knows their text is in purely ASCII ahead of time and they work in UTF8, this information can be used to bit-blast (`memcpy`) the data from UTF8 to ASCII. It is best the standard is given this ability and not require hundreds of users to remake this very basic functionality in [customization points](#).

§ 3.2.3.2. UTF Encodings: variants?

There are many variants of encodings like UTF8 and UTF16. These include [\[wtf8\]](#) or [\[cesu8\]](#) and are useful for internal processing and interoperability with certain systems, like direct interfacing with Java or communication with an Oracle database. However, almost none of these are publicly recommend as interchange formats: both CESU-8 and WTF-8 are documented and used internally for legacy reasons. In some cases, they also represent security vulnerabilities if they are used in interchange for the internet. This makes them less and less desirable to provide VIA the standard. However, it is worth acknowledging that supporting WTF-8 and CESU-8 as encodings will ease individuals who need to roll such encodings for their applications.

More pressingly, there is a wide body of code that operates with `char` as the code unit for their UTF8 encodings. This is also subtly wrong, because on a handful of systems `char` is not unsigned, but signed. Math and bit characteristics for these types are wrong for the typical operations performed in UTF8 encoders and decoders (and many people -- including Markus Scherer that spends a lot of time with ICU -- just wish `char` was unsigned since it would have saved a lot of time from bugs). On one hand, providing variants that allow someone to pick something like the code unit for UTF16 or UTF8 would make it easier to have text types which play nice with the Windows APIs or existing code bases. The interface would look something like this...

```

namespace std { namespace text {

    template <typename CharT, bool encode_null, bool encode_lone_surrogates>
    class basic_utf8;

    using utf8 = basic_utf8<char8_t, false, false>;

    template <typename CharT, bool allow_lone_surrogates>
    class basic_utf16;

    using utf16 = basic_utf8<char16_t, false>;

}}

```

And externally, libraries and applications could add their own using statements and type definitions for the purposes of internal interoperation:

```

namespace my_app {

    using compat_utf8 = std::basic_utf8<char, false, false>;
    using mutf8 = std::basic_utf8<char8_t, true, false>;
    using filesystem16 = std::basic_utf16<wchar_t, true>;

}

```

There is clear utility that can be had here. But, this is not going to be looked into too deeply for the first iterations of this proposal. If there is a need, users are strongly encouraged to chime in (speak up) quickly so that this feature can be added to the proposal before later progression stages.

Finally, there is a plan that for early C++26, the full gamut of WHATWG encodings will be added to the standard, since this covers the minimal viable set of encodings that is required for communicating across the internet and through messaging mediums such as e-mail successfully.

§ 3.2.3.3. Encoding Schemes: Byte-Based

Unicode specifies what are called Encoding Schemes for the encodings whose code unit size exceeds a single byte. This is essentially UTF16 and UTF32, of which there is UTF16 Little Endian (UTF16-LE), UTF16 Big Endian (UTF16-BE), UTF32 Little Endian (UTF32-LE), and UTF32 Big Endian

(UTF32-BE). Encoding schemes can be generically handled without creating extremely specific encodings by creating an `encoding_scheme<...>` template. It will look much like so:

```
// header: <encoding>

namespace std { namespace text {

    template <typename Encoding,
              std::endian endianness = std::endian::native,
              typename Byte = std::byte>
    class encoding_scheme;

}}
```

This is a transformative encoding type that takes the source endianness and translates it to the native endianness. It has an identical interface to the `Encoding` type passed in, with the caveat that the `code_unit` member type is the same as `Byte`. The `Byte` type being configurable is important because there are many interfaces which interoperate using `std::byte`, `unsigned char`, and `char` in the ecosystem. Furthermore, others have realized they can get better performance from their code by avoiding aliasing types altogether and using `enum octet : unsigned char {};` with the necessary definitions to make it usable.

All `encoding_scheme` does is call the same `encode` or `decode` function with small wrappers around the passed-in ranges that takes bytes and composes them into the internal `encoding_code_unit_t<Encoding>` type, or when writing out takes an `encoding_code_unit_t<Encoding>` type and writes it out into its byte-based form.

A few SG16 members have frequently advocated that the base input and outputs for all types matching the `Encoding` concept should be byte-based. This paper disagrees with that supposition and instead goes the route of providing this wrapping encoding scheme. The benefit here is flexibility and independence from byte ordering at the `Encoding` level: the `encoding_scheme` becomes the layer at which such a concern is both concentrated and isolated. Now, *no* encoding needs to duplicate its interface at all, while still retaining strong and separately named types that one can perform additional optimization on.

Writing mostly-duplicate encoding object types for `utf16_be`, `utf16_le`, and other such shenanigans is a thorough and fundamental waste of everyone's time.

This direction is far less boilerplate, and has also already seen implementation experience in [\[libogonek\]](#)'s [\[libogonek-encoding_scheme\]](#) type. Users have not complained. It has also proved to be

implementable by simply decomposing the original input/output ranges into their iterators, and wrapping said iterators with a `__detail::byte_iterator<OriginalIterator>`. It has worked well.

§ 3.2.3.4. Default Encodings

For interactions with encodings, there are times when a default encoding may be inferred from input and output types in § 3.3 High Level's functions. Thusly, 2 traits provide defaults that can be overridden by the program:

```
// header: <encoding>

namespace std { namespace text {
    template <typename T>
    using default_code_unit_encoding_t = /* ... */;

    template <typename T>
    using default_code_point_encoding_t = /* ... */;
}}
```

The implementation for the standard will attempt to select one of the following, or fail, for `default_code_unit_encoding_t`:

- `std::text::execution` if `T` is (possibly cv-qualified) `char`.
- `std::text::wide_execution` if `T` is (possibly cv-qualified) `wchar_t`.
- `std::text::utf8` if `T` is (possibly cv-qualified) `char8_t`.
- `std::text::utf16` if `T` is (possibly cv-qualified) `char16_t`.
- `std::text::utf32` if `T` is (possibly cv-qualified) `char32_t`,
`std::text::unicode_code_point`, or `std::text::unicode_scalar_value`.
- `std::text::encoding_scheme<std::text::utf8>` if `T` is (possibly cv-qualified) `std::byte`.
- Otherwise, the program is ill-formed.

For `default_code_point_encoding_t`:

- `std::text::utf8` if `T` is one of (possibly cv-qualified) `std::text::unicode_code_point`, `std::text::unicode_scalar_value` or `char32_t`.

— Otherwise, the program is ill-formed.

§ 3.2.4. Stateful Objects, or Stateful Parameters?

Stateful objects are good for encapsulation, reuse and transportation. They have been proven in many APIs both C and C++ to provide a good, reentrant API with all relevant details captured on the (sometimes opaque) object itself. After careful evaluation, stateful parameter rather than a wholly stateful object for the function calls in encoding and decoding types are a better choice for this low-level interface. The main and important benefits for having the state be passed to the encoding / decoding function calls as a parameter are that it:

- maintains that encoding objects can be cheap to construct, copy and move;
- improves the general reusability of encoding objects by allowing state to be massaged into certain configurations by users;
- and, allows users to set the state in a public way without having to prescribe a specific API for all encoders to do that.

The reason for keeping encoding types cheap is that they will be constructed, copied, and moved a lot, especially in the face of the ranges that SG16 is going to be putting a lot of work into (`std::text::text_view<View, Encoding, ...>` in a future paper, `normalization_view<View, NormalizationForm>` in a future paper, `decode_view<...>/encode_view<...>/transcode_view<...>` in [this paper](#)). Ranges require that they can be constructed in (amortized) constant time; this change allows shifting the construction for what may be potentially expensive state to other places by un-bundling them from `Encoding` object construction.

Consider the case of execution encoding character sets today, which often defer to the current locale. Locale is inherently expensive to construct and use: if the standard has to have an encoding that grabs or creates a `codecvt` or `locale` member, there will be an immediate loss of a large portion of users over the performance drag during construction of higher-level abstractions that rely on the encoding. It is also notable that this is the same [mistake std::wstring_convert shipped with](#) and is one of the largest contributing reasons to its lack of use and subsequent deprecation (on top of its poor implementation in almost every standard library, from the VC++ standard library to libc++).

In contrast, consider having an explicit parameter. At the cost of making a low-level interface take one more argument, the state can be paid for once and reused in many separate places, allowing a user to front-load the state's expenses up-front. It also allows the users to set or get the locale ahead of time and reuse it consistently. It also allows for encoding or decoding operations to be reused or restart in

the cases of interruptible or incomplete streams, such as network reading or I/O buffering. These are potent use cases wherein such a design decision becomes very helpful.

Finally, this paradigm makes it far more obvious to the end user when the state is inseparable from the encoding object itself. This is the case with a theoretical `any_encoding` and `variant_encoding<Encoding0, Encoding1, ..., EncodingN>`. The necessary state cannot be separated from the encoding object itself: that information is secret in the encoding. A full video exploration of the space can be [found here](#). In short: there must be a way to ensure that a user can create an encoding that has state that is erased within the current compile-time framework. This is how we afford those encodings the ability to work without imposing undue burden on the entire system. It is easy to check if the `encoding_state_t<Encoding>` type is the same as the `Encoding` type, and if that is the case make slight adjustments.

§ 3.3. High Level

Working with the lower level facilities for text processing is not a pretty sight. Consider the usage of the [low-level facilities](#) described above:

```

#include <encoding>
#include <iterator>
#include <span>

int main () {
    std::text::unicode_code_point array_output[41]{};
    std::u8string_view input = u8"ΜΑΓ ΓΛΕΣ ΙΤΑΝ, ΝΙ ΜΙΣ ΥΠ ΝΘΑΝ ΒΡΙΓΓΙΨ.";

    std::text::utf8 encoding{};

    std::u8string_view working_input = input;
    std::span<std::text::unicode_code_point> working_output(array_output);
    std::text::default_handler handler{};
    std::text::utf8::state encoding_state{};

    for (;;) {
        auto result = encoding.decode(working_input, working_output,
                                      handler, encoding_state);
        if (result.error_code != encoding_errc::ok) {
            // not what we wanted.
            return -1;
        }
        if (std::empty(result.input)) {
            break;
        }
        working_input = std::move(result.input);
        working_output = std::move(result.output);
    }

    assert(std::u32string_view(array_output) == U"ΜΑΓ ΓΛΕΣ ΙΤΑΝ, ΝΙ ΜΙΣ ΥΠ ΝΘΑΝ ΒΡΙΓΓΙΨ.");

    return 0;
}

```

These low-level facilities -- while powerful and customizable -- do not represent what the average user will -- or should -- be wrangling with. Therefore, the higher-level facilities become incredibly pressing to make these interfaces palatable and sustainable for developers in both the short and long term. Consider the same encoding functionality, boiled down to something far easier to use:

```
std::u32string output = std::text::decode(u8"ΜΑΓ ΓΛΕΣ ἸΤΑΝ, ΝΙ ΜΙΣ ΥΠ ΝΘΑΝ ΒΡΙΓ  
assert(output == U"ΜΑΓ ΓΛΕΣ ἸΤΑΝ, ΝΙ ΜΙΣ ΥΠ ΝΘΑΝ ΒΡΙΓΓΙΨ.");
```

This is much simpler and does exactly the same as the above, without all the setup and boilerplate. Of course, taking only the input and giving the output is too **much** of a simplification, so there are a few overloads and variants that will be offered. Particularly, there needs to be 3 sets of free functions: `decode/decode_into`, `encode/encode_into`, and `transcode/transcode_into`. These are high-level functions that perform essentially what is shown above, but with numerous overloads that default a few parameters in the case where they can be figured out.

Note that, at the core of all these functions, the loop as shown above captures the core of the work. All of these abstractions are built on the 7 basis operations specified in [§ 3.2.3 The Encoding Object](#). Actually getting additional optimizations is, of course, left to the readers and implementers.

§ 3.3.1. Eager Free Functions

The free functions are written in a way to eagerly consume input and output space, unless given an explicit output container which limits its behavior or an error occurs. This is beneficial because many text processing algorithms receive the bulk of their gains by being able to work on multiple code units / code points. Therefore, this layer of the high level API is provided to satisfy the need where input and output space are of little concern.

§ 3.3.1.1. Free Function `decode`

The `decode` free function provides a High Level API for decoding text. It allows performance with some degree of flexibility and customization through its parameters, as well as additional improvements with the use of some ADL customization points. The core loops behaves as follows:

0. Performing an `auto result = encoding.decode_one(...)` call using the current target input and output views.
1. Checking if the return value's error code is `std::text::encoding_errc::ok`, and returning the result early if it is not.
2. Checking `std::ranges::empty(result.input)`, and returning with a result that has `error_code` set to `std::text::encoding_errc::ok` if it is empty.
3. Otherwise, go to 0 and use the `result.input` and `result.output` views.

The surface of the `decode` API is as follows:

```
// header: <encoding>

namespace std { namespace text {

    template <typename Input, typename Output, typename Encoding,
              typename State, typename ErrorHandler>
    constexpr auto decode_into(Input&& input, Encoding&& encoding,
                               Output&& output, ErrorHandler&& error_handler, State& state)

    template <typename Input, typename Encoding, typename Output,
              typename ErrorHandler>
    constexpr auto decode_into(Input&& input, Encoding&& encoding,
                               Output&& output, ErrorHandler&& error_handler);

    template <typename Input, typename Encoding, typename Output>
    constexpr auto decode_into(Input&& input, Encoding&& encoding,
                               Output&& output);

    template <typename Input, typename Output>
    constexpr auto decode_into(Input&& input, Output&& output);

    template <typename Input, typename Encoding,
              typename ErrorHandler, typename State>
    constexpr auto decode(Input&& input, Encoding&& encoding,
                          ErrorHandler&& error_handler, State& state);

    template <typename Input, typename Encoding,
              typename ErrorHandler>
    constexpr auto decode(Input&& input, Encoding&& encoding,
                          ErrorHandler&& error_handler);

    template <typename Input, typename Encoding>
    constexpr auto decode(Input&& input, Encoding&& encoding);

    template <typename Input>
    constexpr auto decode(Input&& input);

}}
```

The order of arguments is chosen based on what users are likely to specify first. In many cases, all that is needed is the input: the encoding can be chosen automatically for the user based on such. For `decode`, the

`std::text::default_code_unit_encoding_t<std::ranges::range_value_t<Input>>` encoding type is picked (see § 3.2.3.4 Default Encodings). Otherwise, the user must specify the `encoding` object to use themselves. The third parameter is the error handler, which is defaulted to a parameter of type `std::text::default_handler`. The fourth parameter is the state that is used to do the conversion. Given a type `UEncoding` which is `std::remove_cvref_t<Encoding>`, by default, the following is passed:

- If `is_encoding_self_state_t<Encoding>` is true, then `encoding.reset_state();` is called and `encoding` is passed as the `State&` parameter to the appropriate overload.
- Otherwise, `encoding_state_t<Encoding>{}` is used as the parameter to the appropriate overload.

The `decode` family of functions returns a

`std::basic_string<encoding_code_unit_t<Encoding>>` after calling `decode_into` with a `std::ranges::unbounded_view<std::back_inserter<...>>` that fills in the `basic_string`. `decode_into` returns a `decode_result<Input, Output, encoding_state_t<Encoding>>`.

Note: in the current running implementation, there are also separate overloads for `decode` that take an extra template parameter at the beginning called `Result`, which allows the user to write e.g. `std::text::decode<std::vector<std::uint32_t>>(...)` and similar. It is not included in this proposal right now but will be added later, for the purposes of allowing different output types with the simpler calls.

§ 3.3.1.2. Free Function `encode`

The `encode` free function provides a High Level API for decoding text. It allows performance with some degree of flexibility and customization through its parameters, as well as additional improvements with the use of some ADL customization points. The core loop behaves as follows:

0. Performing an `auto result = encoding.encode_one(...)` call using the current target input and output views.
1. Checking if the return value's error code is `std::text::encoding_errc::ok`, and returning the result early if it is not.

2. Checking `std::ranges::empty(result.input)`, and returning with a result that has `error_code` set to `std::text::encoding_errc::ok` if it is empty.
3. Otherwise, go to 0 and use the `result.input` and `result.output` views.

The surface of the `encode` API is as follows:

```

// header: <encoding>

namespace std { namespace text {

    template <typename Input, typename Output, typename Encoding,
              typename State, typename ErrorHandler>
    constexpr auto encode_into(Input&& input, Encoding&& encoding,
                              Output&& output, ErrorHandler&& error_handler, State& state)

    template <typename Input, typename Encoding, typename Output,
              typename ErrorHandler>
    constexpr auto encode_into(Input&& input, Encoding&& encoding,
                              Output&& output, ErrorHandler&& error_handler);

    template <typename Input, typename Encoding, typename Output>
    constexpr auto encode_into(Input&& input, Encoding&& encoding,
                              Output&& output);

    template <typename Input, typename Output>
    constexpr auto encode_into(Input&& input, Output&& output);

    template <typename Input, typename Encoding,
              typename ErrorHandler, typename State>
    constexpr auto encode(Input&& input, Encoding&& encoding,
                          ErrorHandler&& error_handler, State& state);

    template <typename Input, typename Encoding,
              typename ErrorHandler>
    constexpr auto encode(Input&& input, Encoding&& encoding,
                          ErrorHandler&& error_handler);

    template <typename Input, typename Encoding>
    constexpr auto encode(Input&& input, Encoding&& encoding);

    template <typename Input>
    constexpr auto encode(Input&& input);

}}

```

For `encode`, a default encoding of

`default_code_point_encoding_t<std::ranges::range_value_t<Input>>` (§3.2.3.4

`Default Encodings`) is picked when no `Encoding` object is provided. For `encode_into` - which takes an output range to write code units into -- the following is done:

- If `std::is_same_v<typename std::iterator_traits<std::ranges::range_iterator_t<Output>>::iterator_category, std::output_iterator_tag>` is false, `default_code_unit_encoding_t<std::ranges::range_value_t<Output>>{}` is used.
- Otherwise, if the iterator category of the iterators of the output range are `std::output_iterator_tags`, `default_code_point_encoding_t<std::ranges::range_value_t<Input>>{}` is used.

Otherwise, the user must specify the `encoding` object to use themselves. The third parameter is the error handler, which is defaulted to a parameter of type `std::text::default_handler`. The fourth parameter is the state to be used. If it is not provided, then the following is used:

- If `is_encoding_self_state_t<Encoding>` is true, then `encoding.reset_state();` is called and `encoding` is passed as the `State&` parameter to the appropriate overload.
- Otherwise, `encoding_state_t<Encoding>{}` is used as the parameter to the appropriate overload.

The `encode` family of functions returns a

`std::basic_string<encoding_code_unit_t<Encoding>>` after calling `encode_into` with a `std::ranges::unbounded_view<std::back_inserter<...>>` that fills in the `basic_string`. `encode_into` returns a `encode_result<Input, Output, encoding_state_t<Encoding>>`.

Note: in the current running implementation, there are also separate overloads for `encode` that take an extra template parameter at the beginning called `Output`, which allows the user to write e.g. `std::text::decode<std::vector<uint8_t>>(...)` and similar. It is not included in this proposal right now but will be added later, for the purposes of allowing different output types with the simpler calls.

§ 3.3.1.3. Free Function `transcode`

The `transcode` free function provides a High Level API for transforming text from one encoding to another. It allows performance with some degree of flexibility and customization through its parameters, as well as additional improvements with the use of some ADL customization points. The core loop behaves as follows:

0. Performing an `auto d_result = from_encoding.decode_one(...)` call using the current input view and an intermediate temporary output of `encoding_code_point_t<FromEncoding> intermediate[FromEncoding::max_code_points];`.
1. Checking if the return value's error code is `std::text::encoding_errc::ok`, and returning the result early if it is not.
2. Performing an `auto e_result = to_encoding.encode_one(...)` call using the previous temporary `intermediate` output wrapped in a view as the input and the target output view.
3. Checking if the return value's error code is `std::text::encoding_errc::ok`, and returning the result early if it is not.
4. Checking `std::ranges::empty(d_result.input)`, and returning with a result that has `error_code` set to `std::text::encoding_errc::ok` if it is empty.
5. Otherwise, go to 0 and use the `d_result.input` and `e_result.output` views.

The surface of the `transcode` API is as follows:

```
// header: <encoding>
```

```
namespace std { namespace text {
```

```
    template <typename Input, typename FromEncoding,  
              typename Output, typename ToEncoding, typename FromErrorHandler,  
              typename ToErrorHandler, typename FromState, typename ToState>  
    constexpr auto transcode_into(Input&& input, FromEncoding&& from_encoding,  
                                  Output&& output, ToEncoding&& to_encoding,  
                                  FromErrorHandler&& from_error_handler, ToErrorHandler&& to_error_handler,  
                                  FromState& from_state, ToState& to_state);
```

```
    template <typename Input, typename FromEncoding,  
              typename Output, typename ToEncoding, typename FromErrorHandler,  
              typename ToErrorHandler, typename FromState>  
    constexpr auto transcode_into(Input&& input, FromEncoding&& from_encoding,  
                                  Output&& output, ToEncoding&& to_encoding,  
                                  FromErrorHandler&& from_error_handler,  
                                  ToErrorHandler&& to_error_handler, FromState& from_state);
```

```
    template <typename Input, typename FromEncoding,  
              typename Output, typename ToEncoding,  
              typename FromErrorHandler, typename ToErrorHandler>  
    constexpr auto transcode_into(Input&& input, FromEncoding&& from_encoding,  
                                  Output&& output, ToEncoding&& to_encoding,  
                                  FromErrorHandler&& from_error_handler, ToErrorHandler&& to_error_handler);
```

```
    template <typename Input, typename FromEncoding,  
              typename Output, typename ToEncoding, typename FromErrorHandler>  
    constexpr auto transcode_into(Input&& input, FromEncoding&& from_encoding,  
                                  Output&& output, ToEncoding&& to_encoding,  
                                  FromErrorHandler&& from_error_handler);
```

```
    template <typename Input, typename Output, typename ToEncoding,  
              typename FromEncoding>  
    constexpr auto transcode_into(Input&& input, Output&& output,  
                                  FromEncoding&& from_encoding, ToEncoding&& to_encoding);
```

```
    template <typename Input, typename Output, typename ToEncoding>  
    constexpr auto transcode_into(Input&& input, Output&& output, ToEncoding&& to_encoding);
```

```
    template <typename Input, typename FromEncoding,  
              typename ToEncoding, typename FromErrorHandler,
```

```

        typename ToErrorHandler, typename FromState, typename ToState>
constexpr auto transcode(Input&& input, FromEncoding&& from_encoding,
                          ToEncoding&& to_encoding, FromErrorHandler&& from_error_handler,
                          ToErrorHandler&& to_error_handler, FromState& from_state,
                          ToState& to_state);

template <typename Input, typename FromEncoding,
          typename ToEncoding, typename FromErrorHandler,
          typename ToErrorHandler, typename FromState>
constexpr auto transcode(Input&& input, FromEncoding&& from_encoding,
                          ToEncoding&& to_encoding, FromErrorHandler&& from_error_handler,
                          ToErrorHandler&& to_error_handler, FromState& from_state);

template <typename Input, typename FromEncoding,
          typename ToEncoding, typename FromErrorHandler,
          typename ToErrorHandler>
constexpr auto transcode(Input&& input, FromEncoding&& from_encoding,
                          ToEncoding&& to_encoding, FromErrorHandler&& from_error_handler,
                          ToErrorHandler&& to_error_handler);

template <typename Input, typename FromEncoding,
          typename ToEncoding, typename FromErrorHandler>
constexpr auto transcode(Input&& input, FromEncoding&& from_encoding,
                          ToEncoding&& to_encoding, FromErrorHandler&& from_error_handler);

template <typename Input, typename ToEncoding, typename FromEncoding>
constexpr auto transcode(Input&& input, FromEncoding&& encoding, ToEncoding&& to_encoding);

template <typename Input, typename ToEncoding>
constexpr auto transcode(Input&& input, ToEncoding&& encoding);

}}

```

For `transcode`, a default encoding of `default_code_point_encoding_t<std::ranges::range_value_t<Input>>` (§ 3.2.3.4 [Default Encodings](#)) is picked when no `FromEncoding` object is provided. For `transcode_into` -- which takes an output range to write code units into -- the following is done:

- If `std::is_same_v<typename std::iterator_traits<std::ranges::range_iterator_t<Output>>::iterator_category, std::output_iterator_tag>` is false,

`default_code_point_encoding_t<std::ranges::range_value_t<Output>>{}>` is used.

— Otherwise, if the iterator category of the iterators of the output range are

`std::output_iterator_tags`,

`default_code_point_encoding_t<std::ranges::range_value_t<Input>>{}>` is used.

Otherwise, the user must specify the `encoding` object to use themselves. The third parameter is the error handler, which is defaulted to a parameter of type `std::text::default_handler`. The fourth parameter is the state to be used. If it is not provided, given a type `UEncoding` which is `std::remove_cvref_t<Encoding>` then the following is used:

— If `is_encoding_self_state_t<Encoding>` is true, then `encoding.reset_state()`; is called and `encoding` is passed as the `State&` parameter to the appropriate overload.

— Otherwise, `encoding_state_t<Encoding>{}>` is used as the parameter to the appropriate overload.

The `transcode` family of functions returns a

`std::basic_string<encoding_code_unit_t<ToEncoding>>` after calling `transcode` with a `std::ranges::unbounded_view<std::back_inserter<...>>` that fills in the `basic_string`.

Note: in the current running implementation, there are also separate overloads for `transcode` that take an extra template parameter at the beginning called `Output`, which allows the user to write e.g. `std::text::transcode<std::vector<uint16_t>>(...)` and similar. It is not included in this proposal right now but will be added later, for the purposes of allowing different output types with the simpler calls.

§ 3.3.1.4. Free Function `validate`

The `validate` free function provides a High Level API for checking that a range of text is properly in the encoding provided by the user. It's default core implementation works by:

0. Performing an `auto result = encoding.decode_one(...)` call on the input into an intermediate buffer.
1. Checking if an error occurred, and returning failure if so.
2. Performing an `auto intermediate_result = encoding.encode_one(...)` call on a view wrapping the intermediate buffer to the output.

3. Checking if an error occurred, and returning failure if so.
4. Performing a `std::equals` call on the final result, comparing it to the original input consumed.
5. If it is not equal, return failure.
6. If `std::ranges::empty(result.input)`, return true.
7. Go to 0.

The function signature for `validate` is a little different than the above functions that actually do the transcoding. Specifically, this function needs 2 states, one for the `decode_one` call and one for the `encode_one` call. This is problematic for [potential stateful encodings](#), but for most other encodings this is fine.

```
// header: <encoding>

namespace std { namespace text {

    template <typename Input, typename Encoding,
              typename DecodeState, typename EncodeState>
    constexpr auto validate(Input&& input, Encoding&& encoding,
                           DecodeState& decode_state, EncodeState& encode_state);

    template <typename Input, typename Encoding,
              typename DecodeState>
    constexpr auto validate(Input&& input, Encoding&& encoding, DecodeState& decode_state);

    template <typename Input, typename Encoding>
    constexpr bool validate(Input&& input, Encoding&& encoding);

    template <typename Input>
    constexpr bool validate(Input&& input);

}}

```

The order of arguments is chosen based on what users are likely to specify first. In many cases, all that is needed is the input: the encoding can be chosen automatically for the user based on such. For `validate`, the `std::text::default_code_unit_encoding_t<std::ranges::range_value_t<Input>>` encoding type is picked (see [§ 3.2.3.4 Default Encodings](#)). Otherwise, the user must specify the `encoding` object to use themselves. The third parameter is the state, which is passed as follows:

- If `is_encoding_self_state_t<Encoding>` is true, then `encoding.reset_state();` is called and `encoding` is passed as the `State&` parameter to the appropriate overload.
- Otherwise, `encoding_state_t<Encoding>{}` is used as the parameter to the appropriate overload.

Interestingly, we come to a conundrum here with "self-referential" encodings. We cannot use the `encoding` a second time and call `.reset_state()` on it again, nor can we create one from thin air. This means that for `any_encoding/variant_encoding`-style encodings which contain their own state / are stateful, this function will `static_assert(...)` if it cannot work out. There are also hooks as detailed in [§ 3.4.1.3 Customizability: Validating and Counting Free Functions](#).

§ 3.3.1.5. Free Functions `decode_count` and `encode_count`

This proposal will not spoon feed the reader everything: the `decode_count` and `encode_count` functions will be left as an exercise to the reader. (Hint: it's not much different from how the actual encode or decode core default is implemented.)

§ 3.3.2. Safety with the Free Functions

The second problem is the ability to `_lose_` data due to not using lossless encodings. For example, most legacy encodings are lossy when it comes to code points and graphemes outside of their traditional reservoir (e.g., trying to handle Chinese scripts with a latin-1 encoding). Trying to properly encode between these myriad of encodings leaves room for losing information. Even for Wide Character Locale-based (`wchar_t`) data, the only standard transformation to get to UTF32 text requires translating through the normal Character Locale-based (`char`) functions first, leading to loss of information and mojibake (see [A C paper for additional transcoding utilities](#)).

Therefore, an error at compile-time is wanted if a user uses the above high-level free functions, but does not explicitly specify an error handler in the case where a conversion is lossy. Taking an example [from this presentation](#), this puppy emoji cannot fit in ASCII. In general, most Unicode Code Points cannot fit in an ASCII string: this is a dangerous conversion! So, unless you use a non-default error handler, the library will `static_assert` or perform other shenanigans to loudly complain at compile-time:

```

int main (int, char*[]) {
    // Compiler Error: lossy encoding, specify non-default error handler
    std::string ascii_emoji0 = std::text::encode(U"🐶", std::text::ascii_encoding{ });

    // Compiler Error: lossy encoding, specify non-default error handler
    std::string ascii_emoji1 = std::text::encode(U"🐶", std::text::ascii_encoding{ },
        std::text::default_handler{});

    // Okay: you asked for it!
    std::string ascii_emoji2 = std::text::encode(U"🐶", std::text::ascii_encoding{ },
        std::text::replacement_handler{});
    // ascii_emoji2 contains '?'

    // Okay: undefined behavior, but you asked for it.
    std::string ascii_emoji3 = std::text::encode(U"🐶", std::text::ascii_encoding{ },
        std::text::assume_valid_handler{});
    // ascii_emoji3 has no guarantees
    // at this point: undefined behavior was invoked!
}

```

§ 3.3.3. Improving Usability for Low-Memory Environments: Ranges

One of the biggest problems with `std::text::encode(_into)`, `std::text::decode(_into)`, and `std::text::transcode(_into)` is exactly their eager consumption. The defaults for these APIs will create owning containers by default of `std::basic_string<code_unit>/std::basic_string<code_point>` and fill them up as much as they possibly can. This makes these High Level free functions untenable for users in memory-constrained environments. The C++ standard is meant to serve everyone, both high-performance and memory-constrained environments. Therefore, lazy ranges are required to provide low-footprint encode, decode, and transcode operations to everyone.

Most importantly, wrappers around other ranges are employed here. This is important: nobody has time to rewrite all of this functionality just because the API strongly mixed `std::basic_string_view` concerns with encoding concerns. There are spans, string views, and other things outside of the standard that are perfectly suitable for iterating over code units: excluding them by not having this be a wrapper type is a non-starter for getting these abstractions wide adoption in the ecosystem.

§ 3.3.3.1. `decode_view` and `decode_iterator`

`decode_view<Encoding, Range, ErrorHandler, State>` is a templated type that takes the for loop found in [§ 3.3 High Level](#) and turns it into a one-by-one, iterative process that produces iterators as powerful as the iterator category/concept of the `Range` type it is supplied with. It is also meant to work with `std::reference_wrapper`s of `Encoding`, `Range`, `ErrorHandler` and `State` types (to allow views to be instantiated over pre-existing Encodings and Ranges and used to make algorithms work). `decode_iterator<Encoding, Range, ErrorHandler, State>` is also specified as well:


```
// header: <encoding>
```

```
namespace std { namespace text {
```

```
template <typename _Encoding,
          typename Range = basic_string_view<encoding_code_unit_t<_Er
          typename ErrorHandler = default_handler,
          typename State = encoding_state_t<_Encoding>>
class decode_iterator;
```

```
template <typename _Encoding,
          typename Range = basic_string_view<encoding_code_unit_t<_Encoding>,
          typename ErrorHandler = default_handler,
          typename State = encoding_state_t<_Encoding>>
```

```
class decode_view {
public:
```

```
using iterator = decode_iterator<Encoding, Range, ErrorHandler, S>
```

```
using sentinel = decode_sentinel;
```

```
using range_type = Range;
```

```
using encoding_type = Encoding;
```

```
using error_handler_type = ErrorHandler;
```

```
using encoding_state_type = encoding_state_t<encoding_type>
```

```
constexpr decode_view(range_type range) noexcept;
```

```
constexpr decode_view(range_type range, encoding_type encod
```

```
constexpr decode_view(range_type range, encoding_type encod
    error handler type error handler) noexcept;
```

```
constexpr decode_view(range_type range, encoding_type encoding,
                      error_handler_type error_handler, encoding_state_type state) {
```

```
constexpr decode view(iterator it) noexcept;
```

```
constexpr iterator begin() const& noexcept;
```

```
constexpr iterator begin() && noexcept;
```

```
constexpr sentinel end() const noexcept;
```

```
friend constexpr decode_view reconstruct(::std::in_place_ty,
    iterator it, sentinel) noexcept;
```

```
};  
}}
```

The `decode_iterator` produces a `value_type` of `encoding_code_point_t<Encoding>`. It keeps track of how many code points are generated by a call to `encoding.decode_one`, and iterates through however many are present, before calling `encoding.decode_one` again to obtain the next values.

In the case of errors, the standard has a number of well-defined behaviors that prevent the need to add a `.is_valid()` check to the view type, or to provide a `expected`-like wrapper for the `value_type`:

- `default_handler/replacement_handler`: provides replacement characters, which will be inserted into the iteration stream. Errors do not escape and are shown as replacement characters. This works fine.
- `throw_handler`: throws on an error, exceptions escape the `++it` and `*it` calls. This works fine.
- `assume_valid_handler`: user was already invoking UB if errors were hit. This works "fine" (the user asked for it).

Therefore, the only error case wherein `decode_view` and `decode_iterator` perform badly is when the error handler is one which passes through the error without doing anything with the error information with the expectation that the user handles it. The user would be unable to handle it in this case with the custom error handler. There are a few ways to deal with this situation: the first would be to restrict the allowed error handlers into the range and iterator types to Standard Sanctioned™ types. The other would be to just throw hands up when the user passes in an error handler that does not properly throw, message, or handler errors in an appropriate fashion. This proposal currently advocates the latter: passing an error handler to the 4th template parameter is an extreme amount of buy in. If users have gone this far, they must want a very specific custom behavior. Implementations will be encouraged to add asserts to trap users who have poor behavior, but otherwise leave it undefined behavior if errors are not handled for iterator and range types.

Note: This differs from how Tom Honermann's `text_view` and similar behaved. That library returned `Boost.Outcome/std::expected/std::optional`-like result types that one had to further dereference to get to the code points. This represented an ergonomics and a composability problem, because a further transformation step to dereference was always required.

A third option is returning a special type which holds the `decode_one` result and has an implicit conversion to the `code_point` type. It could throw on a conversion where there is an error. This design choice has some serious limitations because it makes `auto` dangerous to use for casual users due to the nature of "magical proxy types". It also forces a throwing of the error on end users, which forces a choice that invalidates the need of environments where exceptions do not exist or are prohibitively expensive.

Note: It is recognized that the Standard does not bless such implementations. This proposal does not care: the needs of C++'s users greatly outweighs the theoretical purity of the C++ abstract machine where the cost of all things is equal and does not matter. The standard's preferred error handling method has a non-zero cost (particularly in binary size) to simply exist that have not been fully optimized into a "do not pay for what you do not use" state. Furthermore, it is still extremely dubious to throw-by-default on any ill-formed text for reasons mentioned above. Therefore, directions wherein the default is equivalent to throwing are not preferred at this time.

§ 3.3.3.2. `encode_view` and `encode_iterator`

This is identical to § 3.3.3.1 `decode_view` and `decode_iterator`, except the name of the view and iterator are `transcode_view` and `transcode_iterator`, respectively as well as a few other minor changes.

- The `Range` template parameter is defaulted to `basic_string_view<encoding_code_point_t<Encoding>>`.
- The `encode_view` view itself produces code units (e.g., `value_type` is `encoding_code_unit_t<Encoding>` rather than code points), one at a time, of the `Encoding` by using `encoding.encode_one`.

Everything else is identical in nature to `decode_view`.

§ 3.3.3.3. `transcode_view` and `transcode_iterator`

This is mostly identical to § 3.3.3.1 `decode_view` and `decode_iterator`, though there are more apparent changes here.

- The name of the view and iterator types are `transcode_view` and `transcode_iterator`, respectively.

- The template parameters are modified to take a `ToEncoding` and a `FromEncoding`, a `ToErrorHandler` and a `FromErrorHandler`, and finally a `ToState` and `FromState`.
- The `Range` template parameter is defaulted to `basic_string_view<encoding_code_unit_t<ToEncoding>>`.
`std::basic_string_view<encoding_code_unit_t<ToEncoding>>`.
- The `value_type` is `encoding_code_unit_t<ToEncoding>` and produces code units, one at a time, of the `ToEncoding`.

Additionally, another important change here is an optimization opportunity. The default implementation of performing a single "`transcode_one`" operation is to:

- Take the input range stored in the class, call `from_encoding.decode_one` with it.
- Take the intermediate output range for the previous `decode_one` call, and feed it into `to_encoding.encode_one`.
- Present the output to the user in a suitable manner.

This is fine, as long as the `code_point` types agree when going from the code units of the `FromEncoding` to the code units of the `ToEncoding`. The problem here is that for many conversions, going from `encoding_code_unit_t<code_unit>` → shared `encoding_code_point_t<FromEncoding>` → `encoding_code_unit_t<ToEncoding>` is an unnecessarily long step. The same way [ADL customization points](#) are provided for the [free functions](#), there must be provisions for turning that through-code-points roundtrip into something a little bit faster.

For example, `ascii` and `utf8` are bitwise compatible. It is extremely foolish to roundtrip that -- for each and every code point/code unit -- through an intermediary `code_point` as is done in the generic core implementation. Therefore, extensibility for this case is provided as described in [§ 3.4.1.1 One-by-one Transcoding Shortcuts](#).

§ 3.4. The Need for Speed

Performance is correctness. If these methods and the resulting interface are not fast enough to meet the needs of the programmers, there will be little to no adoption over current solutions. Thanks to work by Bob Steagall and Zach Laine, it is fact that it is incredibly hard to make a range-based or iterator-based interface which will achieve the text processing speeds that will satisfy users of trivial (`span`-based, pointer-based) need. There are shortcuts when transcoding between certain encoding pairs that should be taken, even in the `code_point`-by-`code_point` transcoding works in the general case.

An explicit goal of this library is that there shall be no room for a lower level abstraction or language here, and the first steps to doing that are recognizing the benefits of eager encoding, decoding and transcoding interfaces, as well as pluggable and overridable behavior for the variety of functionality as it relates to higher-level abstractions.

Research and implementation experience with [\[boost.text\]](#), [\[text_view\]](#) and others has made it plainly clear that while iterators and ranges can produce an extremely efficient binary, it is still not the fastest code that can be written to compete with hand-written/vectorized bulk text processing routines made specifically for each encoding. Therefore, it is imperative that lazy ranges cannot be the only solution. The C++ Standard must steadily and nicely supplant the codebase-specific or ad-hoc solutions individuals keep rolling for encoding and decoding operations.

§ 3.4.1. Speed and Flexibility for Everyone: Customization Points

An important part of that is the ability to provide performance for both lazy, range-based iteration as described in [§ 3.3.3 Improving Usability for Low-Memory Environments: Ranges](#) and fast free functions as described in [§ 3.3.1 Eager Free Functions](#). To this end, an ADL free function scheme similar to the Range Access Customization Points (e.g. `std::ranges::begin` and friends) has been developed to facilitate the customization for speed that users will require for their code.

Considering this is going to be one of the most fundamental text layers that sits between typical text and a lot of the new I/O routines, it is imperative that these conversions are not only as fast as possible, but customizable. The user can already customize the encoding by creating their own conforming encoding object, but encodings still do their transformations on a code point-by-code point basis. Therefore, a means of extensibility needs to be chosen for the `std::text::encode`, `std::text::decode` and `std::text::transcode` ([§ 3.3.1 Eager Free Functions](#)) functions. As this paper is targeting C++23, there exists hope that Matt Calabrese's [\[p1292\]](#) receives favor in the Evolution Design Groups so that the extension mechanisms are simple functions that call simple extension points as laid out below. Failing that, a design similar to `std::ranges`'s customization points -- as laid out in [\[n4381\]](#) -- would be preferred.

What is not negotiable is that it must be extensible. Users should be able to write fast transcoding functions that the standard picks up for their own encoding types. From GB18030 to other ISO and WHATWG encodings, there will always be a need to extend the fast bulk processing of the standard. Current standard library implementers do not have the time to support every single legacy encoding on the planet, and companies do not have the time to petition each and every standard library to add support for their internal encoding. Similarly, government records kept in legacy encodings for political or organizational reasons cannot be locked out of this world either.

Thusly, the following extension points are provided.

§ 3.4.1.1. One-by-one Transcoding Shortcuts

Using the example of `ascii` and `utf8` previously made in this paper, there is room for performing faster one-by-one transcoding. Normally, given a `FromEncoding` and `ToEncoding` such as `ascii` and `utf8` the process involves round-tripping is as follows:

0. Convert input `encoding_code_unit_t<FromEncoding>` → intermediary shared `encoding_code_point_t<FromEncoding>`
1. Convert shared `encoding_code_point_t<FromEncoding>` → `encoding_code_unit_t<ToEncoding>`.

This is accomplished by first calling `.decode_one` on the incoming `input` with an intermediary output, typically an array of `encoding_code_point_t<FromEncoding>` `intermediate_code_points[FromEncoding::max_code_points]`; wrapped up in a view. This intermediary is then put into an `.encode_one` call and the resulting output used for whatever purpose is necessary.

To speed this process up, the free function `text_transcode_one` can be defined by by the user to skip the round trip:

```
// in any related namespace in which ADL can find it

template <typename Input, typename FromEncoding,
          typename Output, typename ToEncoding,
          typename FromErrorHandler, typename ToErrorHandler,
          typename FromState, typename ToState>
std::text::transcode_result<Input, Output, FromState, ToState>
text_transcode_one(Input input, FromEncoding&& from,
                  Output output, ToEncoding&& to,
                  FromErrorHandler&& from_error_handler,
                  ToErrorHandler&& to_error_handler,
                  FromState& from_state, ToState& to_state);
```

The following is a complete example of this customization point.

```

using ascii_to_utf8_result = std::text::transcode_result<
    std::span<char>, std::span<char8_t>,
    std::text::ascii::state, std::text::utf8::state>;

template <typename FromErrorHandler, typename ToErrorHandler>
ascii_to_utf8_result text_transcode_one(std::span<char> input, std::text::a
    std::span<char8_t> output, std::text::utf8& to,
    FromErrorHandler&& from_error_handler, ToErrorHandler&& to_error_ha
    std::text::ascii::state& from_state, std::text::utf8::state& to_sta

    if (input.empty()) {
        // no input: that's fine
        return ascii_to_utf8_result(input, output, from_state, to_s
    }
    if (output.empty()) {
        // error: no room!
        return std::text::propagate_transcode_one_error(from, input
            to, output,
            from_error_handler, to_error_handler,
            from_state, to_state,
            std::text::encoding_errc::insufficient_output_space
            std::span<char, 0>{});
    }
    if ((input[0] & '\x7f') != 0) {
        // error: high bit set in ASCII
        return std::text::propagate_transcode_one_error(from, input
            to, output,
            from_error_handler, to_error_handler,
            from_state, to_state,
            std::text::encoding_errc::invalid_sequence,
            input.subspan<1, 1>());
    }
    // bitwise compatible
    output[0] = static_cast<char8_t>(input[0]);
    // return result
    return ascii_to_utf8_result(input.subspan<1>(), output.subspan<1>()
        from_state, to_state);
}

```

This is faster than the round trip through `unicode_code_point` and requires much less checking and work. When `transcode_view` is, internally, doing the conversion from one code point to

another, it will check if an unqualified call to `text_transcode_one(...)` is valid, and if so call it with its input, output, to/from encoding, and current states.

Note: The function `std::text::propagate_transcode_one_error` takes care of calling the `from_error_handler` and, if appropriate, the `to_error_handler` as well. It does this by constructing a temporary `decode_result` with the current results and a temporary output buffer, milling it through the `from_error_handler`, checking if the temporary output buffer was written into by `from_error_handler`, and passing that intermediary to `to_error_handler` to properly simulate the scheme by which an error would normally be handled in the transcode cycle. This is primarily to facilitate the case when a `std::text::replacement_handler` or similar would communicate a replacement character to the intermediate storage buffer in the default `"encoding_code_unit_t<FromEncoding> → shared encoding_code_point_t<FromEncoding> → encoding_code_unit_t<ToEncoding>"` chain; and, that change needs to be placed in the final output rather than in an intermediate buffer which is going to disappear.

Note: This may be an indication that there should be a third kind of error handler for `transcode`, but that threatens to leak the detail that a `transcode_one` is an optimization of `encode_one` + `decode_one` and make the user sensitive to such an internal optimization.

It is important to note that the above example customization point only works for `std::ranges::contiguous_ranges`; or, anything that can be consumed by the respective `std::span` arguments. This means that a `std::subrange` templated on a `std::list<char>::iterator` would not qualify here, as it is not a contiguous range. This is intentional: there are cases where the kind of range being captured matters for the purposes of optimization. For example, a contiguous range might have its functionality replaced by a function to function calls to the C standard. Only a contiguous range works in that case, because the C standard deals exclusively in pointers.

§ 3.4.1.2. Customizability: Transcoding Free Functions

The free functions are the chance for the user to optimize bulk encoding. This is an area that becomes very important to users all over the world. Many people have already written optimized routines to convert from one encoding to another: it would be a shame if all of this work could not interoperate with the standard as it is. That is why there are 3 ADL-found free functions that are checked for well-formedness, and if so are called by the implementation in `std::text::decode_into`, `std::text::encode_into`, and `std::text::transcode_into`. They are as follows:


```

// in any related namespace in which ADL can find it

template <typename Input,
          typename Encoding, typename Output,
          typename State, typename ErrorHandler>
decode_result<Input, Output, State> text_decode(Input input, const Encoding& encoding,
        Output output, State& state, ErrorHandler&& error_handler);

template <typename Input,
          typename Encoding, typename Output,
          typename State, typename ErrorHandler>
encode_result<Input, Output, State> text_encode(Input input, const Encoding& encoding,
        Output output, State& state, ErrorHandler&& error_handler);

template <typename Input, typename FromEncoding,
          typename Output, typename ToEncoding,
          typename FromState, typename ToState,
          typename FromErrorHandler, typename ToErrorHandler>
transcode_result<Input, Output, FromState, ToState> text_transcode(Input input,
        const FromEncoding& from_encoding, Output output, const ToEncoding& to_encoding,
        FromState& from_state, ToState& to_state,
        FromErrorHandler&& from_error_handler, ToErrorHandler&& to_error_handler);

```

Each of these is the customization hook that a user can write in a namespace to enable a proper conversion from one encoding to another. Nominally, users would use concrete types in place of templated types like `Encoding`, `FromEncoding`, and `ToEncoding`. Because each encoding object is essentially its own "strong object", tags are not required here as the encoding itself acts as an overload-separating, anchoring, strongly-identifying tag that can keep overloads separate and non-clashing. This is different from Boost.Text, where the library must employ [encoding tags](#) on its ranges to gain additional framework-internal optimizations based on [smart tag and type-based dispatching](#). With strong encoding objects, it is not necessary to craft such things internally and, externally, users can rely on it for their ADL extension points:

```

template <typename FromErrorHandler, typename ToErrorHandler>
transcode_result<std::span<char>, std::span<char16_t>,
    win_wrap::windows_1252::state, std::text::utf8::state>
text_transcode(
    std::span<char> input, const win_wrap::windows_1252& encoding,
    std::span<char8_t> output, const std::text::utf16& to_encoding,
    win_wrap::windows_1252::state& from_state,
    std::text::utf16::state& to_state,
    FromErrorHandler&& from_error_handler, ToErrorHandler&& to_error_ha

    if (input.empty()) {
        // do nothing
        return transcode_result</*...*/>(/* ... */);
    }

    int Needed = MultiByteToWideChar(1252, 0,
        input.data(), static_cast<int>(input.size()),
        nullptr, 0);
    if (Needed == 0 || (Needed > static_cast<int>(output.size()))) {
        // handle error ...
        return std::text::propagate_transcode_error(input, output,
            from_handler, to_handler, from_state, to_state,
            std::text::encoding_errc::insufficient_output_space,
            std::span<char, 0>{{}});
    }

    int Succ = MultiByteToWideChar(1252, 0,
        input.data(), static_cast<int>(input.size()),
        reinterpret_cast<wchar_t*>(output.data()), static_cast<int>
    if (Succ == 0) {
        // handle error ...
        return std::text::propagate_transcode_error(input, to_encod
            output, from_encoding,
            transcode_result</*...*/>(/* ... */),
            std::text::encoding_errc::invalid_sequence,
            std::span<char, 0>{{}});
    }
    return transcode_result</*...*/>(/* ... */);
}

```

This does not show all the error handling, but it is a full explanation/demonstration of a custom `windows_1252` encoding defined by a user going through the customization point to get to `utf8` encoded text. Note that this is a slight simplification, since there are additional checks for what kind of error handler is present and whether or not valid substitution can be performed (e.g., since `MultiByteToWideChar` does not accept "unique replacement" characters, but `WideCharToMultiByte` does).

Note: Like in § 3.4.1.1 *One-by-one Transcoding Shortcuts*, the function `std::text::propagate_transcode_error` takes care of calling the `from_error_handler` and, if appropriate, the `to_error_handler` as well.

There does exist some concern for individuals who may want to do specializations for the standard's encodings. The specification will permit someone to write their own `std::text::utf8` $\Rightarrow$ `std::text::utf16` optimization, which will take precedent. This does not let the implementation off the hook for performance: this is only expected to be done for cases where the end-user knows their target architecture better than the standard could (small embedded devices with obscure chipsets and ISAs, and platforms with custom compilers, and similar). Common environments can and absolutely should be optimized by the implementation because there is a bounded set of only 9 possible encodings that the C++ Standard will include at first if this proposal progresses all the way.

**Even if this is possible, it is absolutely expected for implementations to optimize common Unicode encoding pairs with OS or library-internal specific algorithms. If a vendor fails to do this, please file a bug against their implementation.
Loudly.**

§ 3.4.1.3. Customizability: Validating and Counting Free Functions

The `std::text::validate` function also needs a customization point, as well as `std::text::encode_count` and `std::text::decode_count`. To start, there are efficient ways to count code units (e.g., in UTF-8) that do not require synthesizing the full code point value. This can be used to save on speed when counting the size of a very large buffer of text. Similarly, `validate` can be done cheaply and efficiently when compared to the common loop outlined in § 3.3.1.4 *Free Function validate*. Therefore, there are ADL customization points that are as follows:

```

// in any related namespace in which ADL can find it

template <typename Input,
          typename Encoding, typename Output,
          typename State, typename ErrorHandler>
count_result<Input, State> text_decode_count(Input input, const Encoding& e,
      State& state, ErrorHandler&& error_handler);

template <typename Input,
          typename Encoding, typename Output,
          typename State, typename ErrorHandler>
count_result<Input, State> text_encode_count(Input input, const Encoding& e,
      State& state, ErrorHandler&& error_handler);

template <typename Input, typename Encoding,
          typename DecodeState, typename EncodeState>
validate_result<Input, DecodeState> text_validate(Input input, const Encoding& e,
      DecodeState& state, EncodeState& state);

template <typename Input, typename Encoding, typename DecodeState>
validate_result<Input, DecodeState> text_validate(Input input, const Encoding& e,
      DecodeState& state);

```

Notably, there are two `text_validate` functions that can be opted into that take 3 or 4 arguments, respectively. This is for the rare case of an encoding that both cannot create a default state, like ones where `is_self_state_encoding_v<Encoding>` is true (e.g. the `any_encoding/variant_encoding<Enc0, Enc1, ...>` described in this proposal).

In this case, we need a customization point wherein such an encoding, using internal/secret knowledge, can do its validation without needing to rely on the 4-argument `std::text::validate` overload and the core default loop's specification. This satisfies the ability of self-state encodings to escape the need to pass itself twice to the `validate` function.

§ 4. Implementation Experience

There are implementations of this work, taking some of it in part or in full.

§ 4.1. Previous Work

While the ideas presented in this paper have been explored in various different forms, the ideas have never been succinctly composed into a single distributable library. Therefore, the author of this paper is working on an implementation that synthesizes all of the learning from [\[icu\]](#), [\[boost.text\]](#), [\[text_view\]](#) and [\[libgonek\]](#). Reportedly, an implementation using a similar system exists in a few Fortune 500 company codebases. [\[copperspice\]](#) also has a somewhat similar implementation, but differs in a few places.

§ 4.2. Current Work

This paper's r2 hopes to contain benchmarks, initial implementation and usage experience. This paper's r3 hopes to contain more benchmarks, refined implementation and additional field and usage experience after a more valuable and viable minimum product is established. The current implementation is being incubated in a private implementation in [phd.text](#) (please e-mail the author if you would like to access the implementation).

§ 5. FAQ

Some commonly asked questions.

§ 5.1. Question: Why is there a [max_code_points](#) value? Won't you only ever output a single unicode code point?

This is incorrect. There are cases for encodings such as TSCII that output multiple unicode code points at once. The minimum required space must be dictated by the encoding: C++ made the mistake for [basic_filebuf](#) with the [infamous "N:1" rule](#), and that rule is one of the primary reasons file-based streams (which can be any [\(o|i\)stream](#) in an inheritance-based design, as well as nearly anything with the wide use of what file descriptors represent in many operating systems) cannot handle Unicode properly in many implementations (chief among them, Microsoft Windows).

§ 5.2. Question: What about Old Unicode Encodings / Private Use Area Encodings?

These are treated like legacy encodings. Someone must convert to "normal" (Unicode vRight-Now) Unicode in order to have higher level algorithms work. If this includes Private Use Area characters, then a person will need the ability to customize the normalization algorithms for use in getting e.g.

Medieval Text and Biblical Text to normalize properly. This will be covered in a future paper on a `normalize(...)` free function, a `normalization_view` type, and `nf(k)(d/c)/fcc` normalization objects provided by the standard. SG16 at the moment is against trying to create customization points and changes for the Unicode Character Database and give PUA code points different properties. Individuals who use e.g. Unicode v6 w/ Softbank Private Use Area or [TACE 16](#) Encodings will need to convert any Private Use Area characters to Unicode and normalize, or provide their own normalization form for upcoming papers.

§ 5.3. Question: It can be faster to bulk-decode, then bulk-encode instead of one-by-one transcoding. Why not that design?

While this is true, as asserted in the [§ 3.3.1.3 Free Function transcode](#) section, bulk decoding requires that there is a intermediary storage in to bulk-decode into. This imposes an invisible intermediate in the API, or requires explicitly allowing the user to pass one in. Furthermore, a user may only want to partially decode, partially encode, and then repeat because there is some internal memory limit rather than do a single "complete" bulk conversion.

A significant amount of thought and experimental implementation went into potentially providing both a `transcode` function that behaves as is currently specified, PLUS a `decode_encode` function that does a bulk decode and then a bulk encode. The design space was deemed a little too fraught with knobs and potential for exceeding user expectations in unexpected ways. This does not mean a regular user cannot enjoy the benefits of building a similar abstraction. Both the `decode` and `encode` functions are available for a user to apply the right amount of each to achieve a goal similar to the one behind the `decode_encode` abstraction previously envisioned.

§ 5.4. Question: Where is the specification for `normalization_view<nfkc>` and `normalize(...)`?

Normalization is separable from the low-level transcoding, and even though APIs like `MultiByteToWideChar` and similar have additional parameters for doing automatic decomposition or composition upon transcoding, more recently the API has switched to doing these things in 2 separate phases. It is unclear whether there is a performance gain for the two being combined as it is in Windows's APIs, but without such performance data we prefer correctness and existing practice. Furthermore, normalization overloads can always be added to the transcoding interfaces later, if a combined interface proves to have benefits. There is also an open question about the existence of normalization within the highest level abstraction types like `std::text::basic_text` and whether or not those invariants be enforced. Currently, Zach Laine's Boost.Text enforces normalization on creation and insertion of data into

§ 5.5. Question: Where is the specification for `std::text::basic_text` and `std::text::basic_text_view`?

Those types as currently imagined requires additional functionality, like normalization and potentially segmentation algorithms (e.g., for making Grapheme Clusters). It will be split off into a separate paper, even if we allude to its existence and use in this proposal.

§ 6. Acknowledgements

Thanks to R. Martinho Fernandes, whose insightful Unicode quips got me hooked on the problem space many, many years ago and helped me develop my first in-house solution for an encoding container adaptor several years ago. Thanks to Mark Boyall, Xeo, and Eric Tremblay for bouncing off ideas, fixes, and other thoughts many years ago when struggling to compile libogonek on a disastrous Microsoft Visual Studio November 2012 CTP compiler.

Thanks to Tom Honermann, who had me present my second SG16 meeting before it was SG16 and help represent and carry his papers which gave me the drive to help fix the C++ standard for text. Many thanks to Zach Laine, whose tireless implementation efforts have given me much insight and understanding into the complexities of Unicode and whose implementation in Boost.Text made clear the tradeoffs and performance issues. Thanks to Mark Zeren who helped keep me in SG16 and working on these problems.

And thank you to those of you who grew tired of an ASCII-only world and supported this effort.

§ References

§ Informative References

[BOOST.TEXT]

Zach Laine. Boost.Text. October 20th, 2018. URL: <https://github.com/tzlaine/text>

[CESU8]

Unicode Consortium. UTR #26, Compatibility Encoding Scheme for UTF-16: 8-Bit (CESU-8). March 13th, 2019. URL: <https://www.unicode.org/reports/tr26/>

[COPPERSPICE]

CopperSpice C++ Libraries. CsString. March 2nd, 2020. URL: https://github.com/copperspice/cs_string

[FAST-UTF8]

Bob Steagall. [Fast Conversion From UTF-8 with C++, DFAs, and SSE Intrinsics](https://www.youtube.com/watch?v=5FQ87-Ecb-A). September 26th, 2019. URL: <https://www.youtube.com/watch?v=5FQ87-Ecb-A>

[ICU]

Unicode Consortium. [International Components for Unicode](http://site.icu-project.org/). April 17th, 2019. URL: <http://site.icu-project.org/>

[LIBOGONEK]

R. Martinho Fernandes. [Ogonek](https://github.com/libogonek/ogonek). December 9th, 2013. URL: <https://github.com/libogonek/ogonek>

[LIBOGONEK-ENCODING_SCHEME]

R. Martinho Fernandes. [encoding_scheme](https://github.com/libogonek/ogonek/blob/devel/include/ogonek/encoding/encoding_scheme.h%B%2B#L80). December 9th, 2013. URL: https://github.com/libogonek/ogonek/blob/devel/include/ogonek/encoding/encoding_scheme.h%B%2B#L80

[N2440]

JeanHeyd Meneide. [Restartable and Non-Restartable Functions for Efficient Character Conversions](https://thephd.github.io/vendor/future_cxx/papers/source/n2440). March 2nd, 2020. URL: https://thephd.github.io/vendor/future_cxx/papers/source/n2440

[N3574]

Mark Boyall. [Binding stateful functions as function pointers](https://wg21.link/n3574). 10 March 2013. URL: <https://wg21.link/n3574>

[N4381]

Eric Niebler. [Suggested Design for Customization Points](https://wg21.link/n4381). 11 March 2015. URL: <https://wg21.link/n4381>

[P0244R2]

Tom Honermann. [Text_view: A C++ concepts and range based character encoding and code point enumeration library](https://wg21.link/p0244r2). 13 June 2017. URL: <https://wg21.link/p0244r2>

[P1292]

Matt Calabrese. [Customization Point Functions](https://wg21.link/p1292). October 10th, 2018. URL: <https://wg21.link/p1292>

[RANGE-V3]

Eric Niebler; Casey Carter. [range-v3](https://github.com/ericniebler/range-v3). June 11th, 2019. URL: <https://github.com/ericniebler/range-v3>

[SOL2-WSTRING_CONVERT]

ThePhD. [wstring_convert sucks](https://github.com/ThePhD/sol2/issues/571). January 27th, 2018. URL: <https://github.com/ThePhD/sol2/issues/571>

[TEXT_VIEW]

Tom Honermann. [text_view](https://github.com/tahonermann/text_view). November 10th, 2017. URL: https://github.com/tahonermann/text_view

[WTF8]

Simon Sapin. [The WTF-8 encoding](https://simonsapin.github.io/wtf-8/). May 12th, 2018. URL: <https://simonsapin.github.io/wtf-8/>